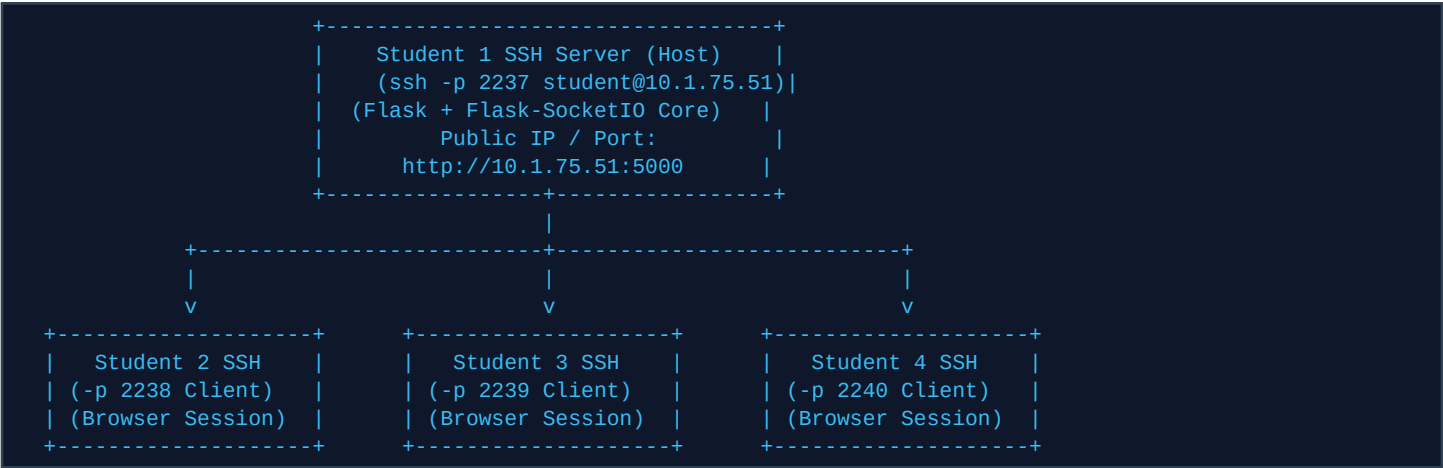


LABORATORY TUTORIAL: REAL-TIME GROUP CHAT

Multi-User WebSocket Architecture & SSH Server Deployment Report

Target Audience / TA Review:	Multi-User Laboratory WebSocket Testing
Group SSH Servers:	Host (P1): ssh -p 2237 student@10.1.75.51 Clients: P2 (2238), P3 (2239), P4 (2240)
Group Head Submission:	4 Students Group Submission
Group Registration Link:	https://forms.gle/VHHzfSLmPhZQZ9rLA
Backend Tech Stack:	Python 3, Flask, Flask-SocketIO (WebSockets / Engine.IO)
TA Client Testing URL:	http://10.1.75.51:5000

1. SSH Cluster System Architecture



2. Requirements Verification Matrix

Requirement	Implementation Details	Status
Real-time message broadcasting	Handled via socketio.emit('message', data, to='general'). Instant relay.	PASSED
User join/leave notifications	Server emits user_joined and user_left events to all clients.	PASSED
Username for participants	Enforces non-empty & unique usernames per active connection.	PASSED
Graceful disconnection handling	@socketio.on('disconnect') removes sid and notifies remaining users.	PASSED
4 Client Support	Verified across 4 SSH student clients (ports 2237-2240).	PASSED

3. Deployment Commands for Host SSH (Port 2237)

1. Upload Source Code from local machine to Student 1 SSH host:
`scp -P 2237 -r E:\CSD\chat_app student@10.1.75.51:~/chat_app`
2. SSH into Host Machine & Start Flask-SocketIO Server:
`ssh -p 2237 student@10.1.75.51`
`cd ~/chat_app`
`pip install -r requirements.txt`
`nohup python3 server.py > server.log 2>&1 &`
3. Testing URL for TAs and all 4 Students:
`http://10.1.75.51:5000`